Incident Response Report

- Incident Response Report — Home SOC Lab
  - 1. Executive Summary
  - 2. Scope and Objective
  - 3. Reconnaissance
  - 4. Exploitation
  - 5. Persistence & Command-and-Control
  - 6. Detection
  - 7. Containment & Eradication
  - 8. Verification / Recovery
  - 9. Indicators of Compromise (IOCs)
  - 10. Lessons Learned
  - 11. Evidence Notes & Known Gaps
  - 12. Lab Architecture
  - Appendix: Full Timeline

Incident Response Report — Home SOC Lab

Classification: Simulated intrusion exercise (self-hosted lab) **Environment:** Isolated Host-Only virtual network **Analyst:** Daniel Diaz **Date of exercise:** September 2026

---

1. Executive Summary

This report documents a full incident-response cycle carried out in a controlled home lab, built to simulate a realistic network intrusion from initial compromise through detection, containment, and eradication.

A vulnerable Linux target (Metasploitable2) was compromised via a known remote code execution vulnerability, after which an attacker-style persistence mechanism combined with a Command-and-Control (C2) beacon was installed. A SIEM stack (Wazuh) paired with a network intrusion detection system (Suricata) was used to detect the recurring C2 traffic, correlate it into a custom alert rule, and confirm the compromise. The threat was then contained and remediated, with post-remediation monitoring supporting that no further malicious activity was generated.

The exercise validates an end-to-end blue-team workflow: **Reconnaissance → Exploitation → Persistence/C2 → Detection → Containment → Eradication → Verification.**

---

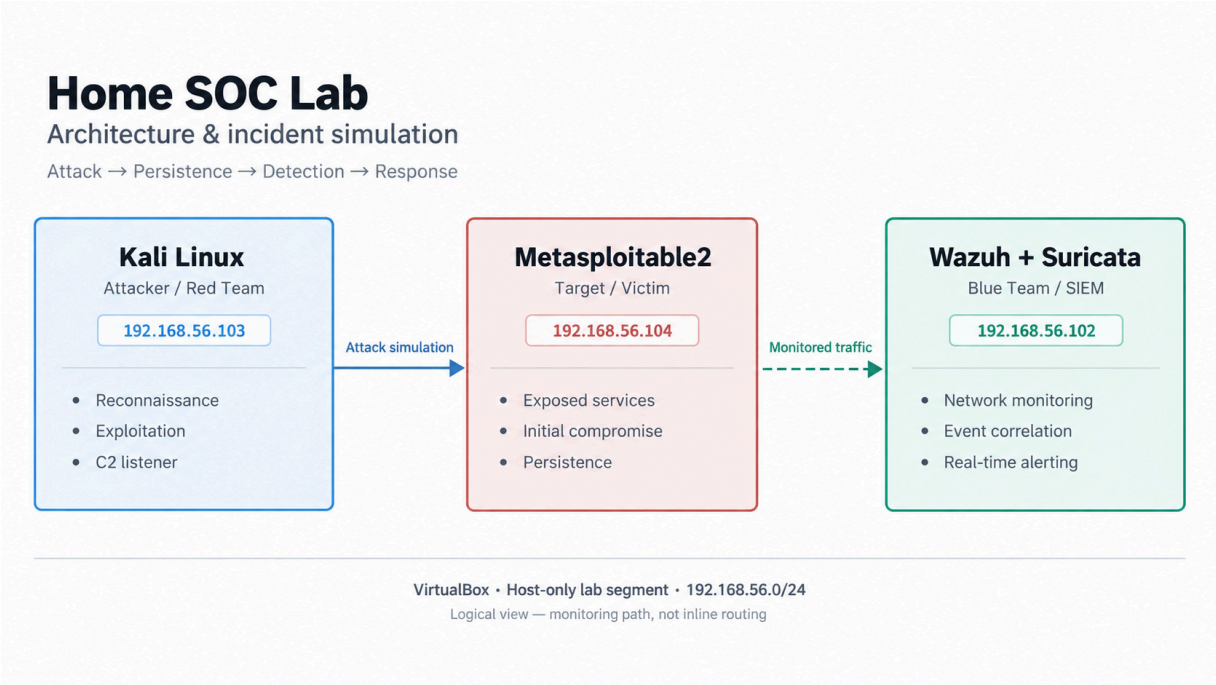
2. Scope and Objective

Objective: Simulate a realistic single-host compromise and demonstrate the ability to detect, respond to, and remediate it using a self-built SIEM/NIDS stack, mirroring the workflow of a SOC analyst handling a live incident.

Lab topology:

Role	System	IP Address
Attacker / Red Team	Kali Linux	192.168.56.103
Target / Victim	Metasploitable2	192.168.56.104
SIEM / Blue Team	Wazuh Server + Suricata	192.168.56.102

All hosts reside on a VirtualBox Host-Only network (`192.168.56.0/24`), which by design carries no route to the internet. The Wazuh server observes traffic on this segment for detection purposes; it does not act as a mandatory gateway between the attacker and target hosts.



Home SOC Lab Architecture

Logical architecture diagram. This is a simplified, documentary representation of the lab built for this report — not a live network capture.

---

3. Reconnaissance

An `nmap` service scan (`nmap -sV -sC`) was run against the target (192.168.56.104) to enumerate exposed services prior to exploitation.

Key findings:

- vsftpd 2.3.4 (port 21) — anonymous FTP access enabled
- Samba smbd 3.0.20-Debian (ports 139/445)
- UnrealIRCd (port 6667)
- Java RMI Registry (port 1099)
- A root shell exposed directly on port 1524
- Multiple other outdated, unpatched services typical of an intentionally vulnerable host

Samba's `usermap_script` misconfiguration was selected as the exploitation vector, prioritized over the more commonly-referenced vsftpd 2.3.4 backdoor for its technical relevance and direct impact (remote code execution as root, no privilege escalation required).

---

4. Exploitation

Vulnerability: Samba “username map script” Command Execution (**CVE-2007-2447**) **Method:** Metasploit module `exploit/multi/samba/usermap_script`, payload `cmd/unix/reverse`

The exploit was launched against 192.168.56.104, establishing a reverse command shell back to the attacker host on 192.168.56.103. Access was confirmed as **root**, with no privilege escalation step required:

```
whoami        → root
id            → uid=0(root) gid=0(root)
uname -a      → Linux metasploitable 2.6.24-16-server
```

Timestamp of initial compromise: 2026-09-14, 17:19:43

---

5. Persistence & Command-and-Control

To simulate realistic post-exploitation attacker behavior, a lightweight persistence and C2 mechanism was installed rather than a static backdoor user, combining persistence and beaconing into a single technique.

Mechanism: - A cron entry was added to `/etc/crontab` on the target, executing a shell script (`/tmp/beacon.sh`) every 2 minutes. - The script opened a reverse shell connection back to the attacker host (192.168.56.103) on a dedicated listener port, simulating periodic outbound C2 “check-in” traffic — a common real-world beaconing pattern used by malware and post-exploitation frameworks.

Timestamp of persistence establishment / first confirmed beacon: 2026-09-14, 21:51:42

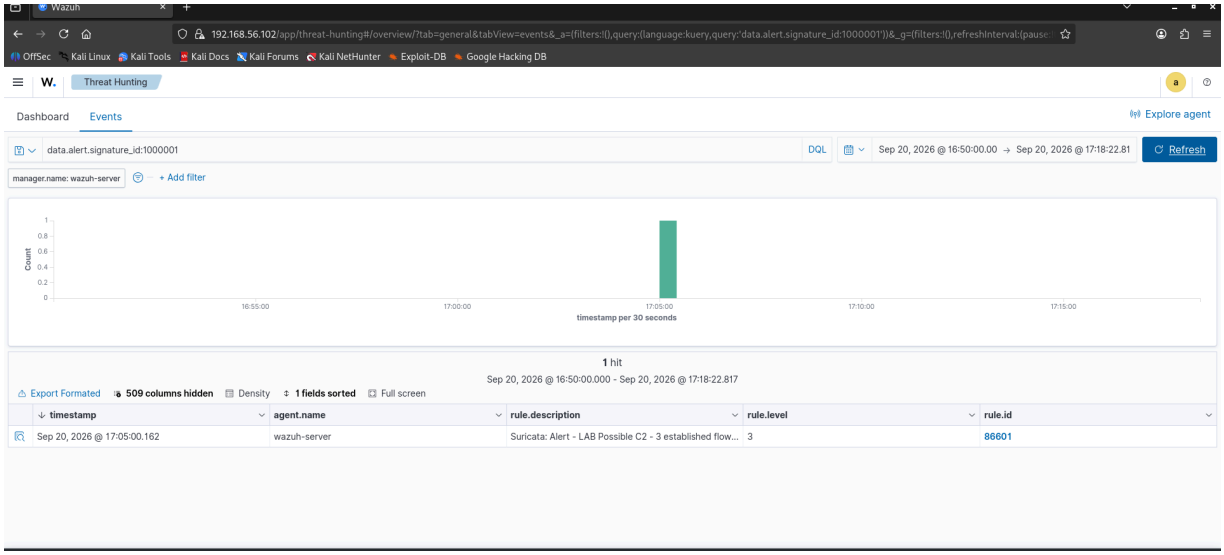
This periodic, low-and-slow connection pattern was deliberately chosen as the detection target for the following phase, since beaconing behavior is one of the most reliable network-based indicators of an active compromise.

---

6. Detection

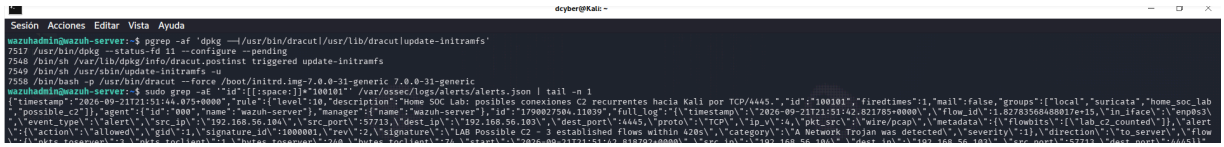
Tools: Suricata (network IDS) + Wazuh (SIEM / log correlation)

A custom Suricata signature was developed to flag repeated short-lived TCP connections from the target host to the attacker’s listener port within a short time window — the network signature of a beaconing implant.



Suricata alert detected in Wazuh

Suricata writes its alerts to `eve.json`, which the Wazuh manager's log collector reads as JSON (a `<localfile>` block in `ossec.conf`). A custom correlation rule (**rule ID 100101**, severity level 10) was authored on top of Wazuh's built-in Suricata rule to flag this signature as a probable C2 channel. The exact rule files are published in [`detections/`](detections/).

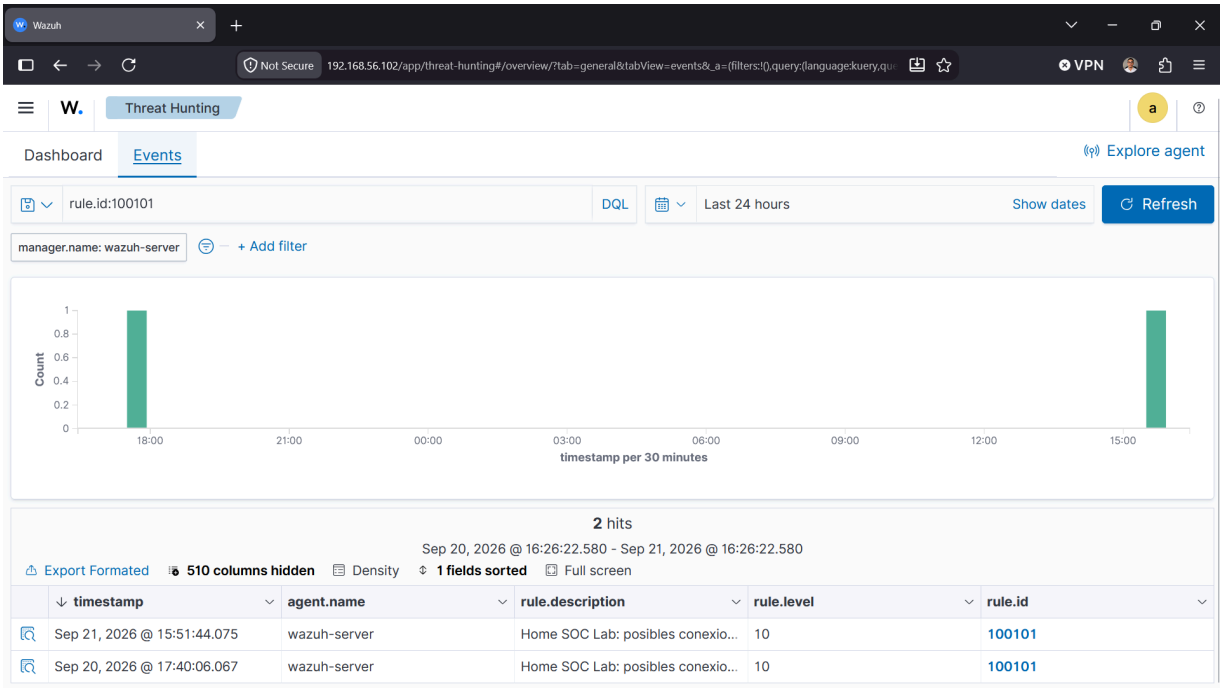


Wazuh dashboard showing correlated alert detail

Confirmed detections:

Dashboard time (UTC−6)	UTC	Rule	Level	Description
2026-09-20, 17:40:06	2026-09-20, 23:40:06	100101	10	Home SOC Lab: possible recurring C2 connections
2026-09-21, 15:51:44	2026-09-21, 21:51:44	100101	10	Home SOC Lab: possible recurring C2 connections

The dashboard shows times in the browser's time zone (UTC−6); the raw alert in `alerts.json` carries the UTC value (`2026-09-21T21:51:44.075+0000` for the second detection). The first raw Suricata alert for the signature (Wazuh rule 86601, `evidence/01-suricata-alert-detected.png`) is timestamped 2026-09-20 17:05:00 UTC−6 (23:05:00 UTC).



Wazuh dashboard confirming two correlated detections

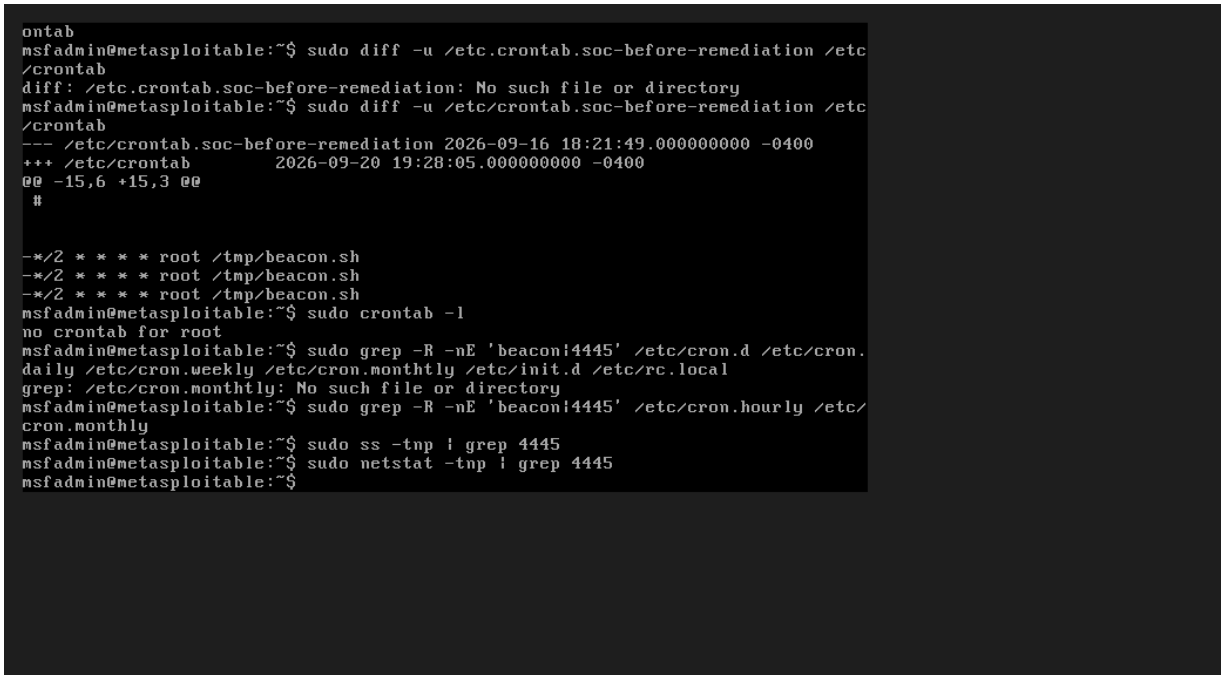
The alert firing on two separate days supports that the detection logic matched the beacon traffic more than once rather than being a one-off. Note: a system clock discrepancy on the Wazuh server was identified and corrected during this exercise (see *Lessons Learned*), and the size of the offset was not recorded. Both detections are timestamped *after* the remediation in Section 7 when the clocks are compared as recorded; Section 8 explains what this does and does not allow us to conclude.

---

7. Containment & Eradication

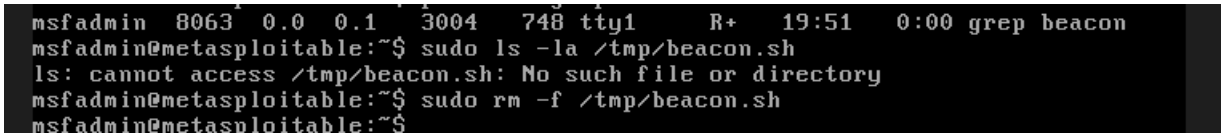
Once detection was validated, the compromised host was remediated:

1. **Removed persistence:** The three malicious cron entries referencing `/tmp/beacon.sh` were removed from `/etc/crontab` using `sed`. A copy of the crontab was preserved before editing, and the change was verified with both a `diff` against that backup and a hash comparison of the file before and after the edit.



Crontab diff confirming removal of persistence entries

2. **Checked for additional persistence:** Searched `/etc/cron.d`, `/etc/cron.daily`, `/etc/cron.weekly`, `/etc/cron.hourly`, `/etc/cron.monthly`, `/etc/init.d`, and `/etc/rc.local` for any reference to the beacon or its listener port — no matches found. The root user was confirmed to have no personal crontab.



Verification of no active beacon process or additional cron persistence

3. **Verified no active process:** Confirmed via `ps aux` that no `beacon.sh` process was running in memory.
4. **Verified no active connection:** Confirmed via `ss` and `netstat` that no TCP connection or listener remained on the C2 port.
5. **Verified artifact removal:** Confirmed the `/tmp/beacon.sh` script no longer existed on disk at the time of verification. This confirms the artifact's absence at that point in time; it does not by itself establish the exact moment the beacon process last executed.

Timestamp of remediation: 2026-09-20, 19:28:05 −0400 on the Metasploitable2 clock, i.e. 23:28:05 UTC / 17:28:05 UTC−6 (modification time of `/etc/crontab`, shown in the `diff -u` header in the screenshot above)

---

8. Verification / Recovery

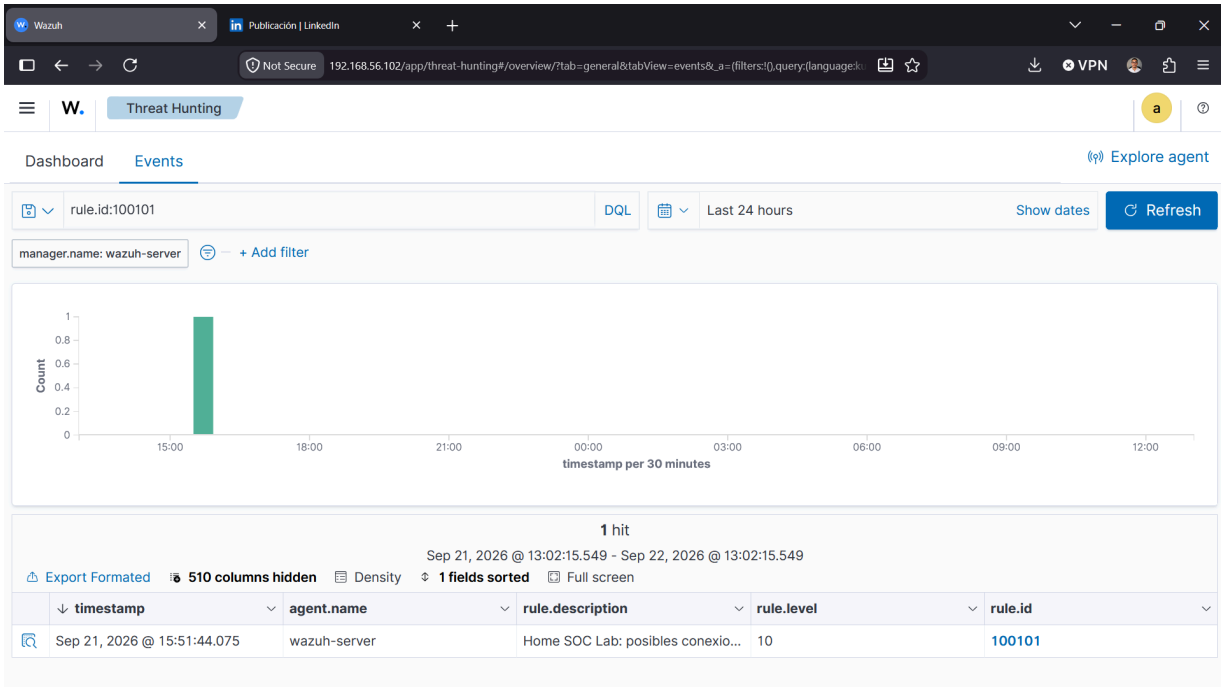
Following remediation, the Wazuh dashboard was monitored over a 24-hour window (2026-09-21 13:02 → 2026-09-22 13:02 UTC−6). One rule 100101 alert appeared in that window, at 2026-09-21 15:51:44 UTC−6, and no further alerts followed it for the remaining ~21 hours.

Timeline inconsistency. Normalized to UTC, the evidence places the events in this order:

UTC	Event	Clock
2026-09-20 23:05:00	First raw Suricata alert for `sid:1000001` (rule 86601)	Wazuh server
2026-09-20 23:28:05	`/etc/crontab` modified (remediation)	Metasploitable2
2026-09-20 23:40:06	First rule 100101 alert	Wazuh server
2026-09-21 21:51:44	Second rule 100101 alert	Wazuh server

Taken at face value, both rule 100101 alerts came after the persistence was removed, which should not happen if the cron entry was the only thing running the beacon. Two explanations fit the evidence: (a) the Wazuh server clock was still wrong when these events were recorded, so the ordering across the two hosts is unreliable; or (b) the beacon kept running for a while after the crontab edit. The evidence collected does not settle which one is true.

What the evidence does support: no TCP/4445 connection or `beacon.sh` process remained on the target at verification time (Section 7), and no alert was generated after 2026-09-21 21:51:44 UTC for the rest of the monitoring window. This supports that the C2 channel was eventually eradicated, but the exact moment it stopped cannot be tied to the remediation timestamp. In a real investigation this would be escalated as an open question and settled by checking NTP status on both hosts before closing the case.



Dashboard confirming no new alerts after remediation

9. Indicators of Compromise (IOCs)

Type	Indicator
Persistence mechanism	Cron entry in `/etc/crontab` executing a script every 2 minutes
Dropped file	`/tmp/beacon.sh`
Network behavior	Recurring short-lived outbound TCP connections at ~2-minute intervals to a fixed external host/port
Detection rule	Wazuh custom rule ID `100101`, Suricata custom signature (network trojan / C2 category)

(Specific payload contents, exact listener port, and full script logic are intentionally omitted from this report to avoid providing a directly reusable attack script.)

For a technique-by-technique MITRE ATT&CK mapping of this incident, including detection coverage and gaps, see `mitre_attack_mapping.md`.

---

10. Lessons Learned

- **Beaconing is a reliable detection surface.** Even a simple, low-volume periodic connection pattern was sufficient to build a working correlation rule — this mirrors how many real-world C2 frameworks are caught in production environments.
- **Systemd timeouts matter at scale.** During lab setup, both Suricata and the Wazuh indexer (OpenSearch) failed to start under default systemd timeouts due to large rule sets / JVM startup time — a reminder that infrastructure tuning is as much a part of SOC operations as detection logic itself.
- **Log pipeline visibility gaps are easy to miss.** Suricata alerts initially never reached the Wazuh dashboard due to a missing `<localfile>` block and a missing JSON decoder — a good reminder to always verify the full data pipeline end-to-end, not just that the source tool is generating logs.
- **Clock accuracy affects evidence quality.** A system clock discrepancy on the Wazuh server was found and corrected mid-exercise. This was a useful reminder that in real SOC operations, unsynchronized clocks across hosts can distort timelines and undermine confidence in correlated evidence — NTP synchronization should be a baseline check, not an afterthought.
- **Verification closes the loop.** Remediation isn't complete without gathering evidence that the malicious activity actually stopped — the post-remediation monitoring window was as important as the fix itself.

---

11. Evidence Notes & Known Gaps

In the interest of accuracy, this report documents the following limitations honestly rather than overstating the evidence collected:

- The architecture diagram is a documentary reconstruction built to illustrate the lab's logical layout; it is not a live configuration export.
- Timestamps come from three different clocks: Metasploitable2 (UTC−4), the Wazuh server (raw alerts in UTC) and the dashboard (UTC−6). The Wazuh server clock was corrected mid-exercise without recording the offset, which is why the order of detection and remediation cannot be confirmed (Section 8). The time zone of the attacker-side timestamps from 2026-09-14 was not recorded.

- The exploitation phase (Section 4) is described from command history and session output; a full terminal recording of the live Metasploit session was not preserved and is not included as a screenshot in this version.
- Post-remediation TCP connection checks were performed as described in Section 7, but a final independent re-check immediately before closing this report was not separately re-captured.
- The AI-assisted alert triage pipeline was built and tested after this incident was closed (2026-09-28 to 2026-09-29), using alerts based on this incident's detections. It played no part in the detection or response described in this report. Its design, red-team results and known limitations are documented in `AI/README.md` and `pipeline_metrics.md`.

These gaps are called out deliberately: an incident report that only shows favorable evidence is less credible than one that is transparent about what was and wasn't captured.

---

12. Lab Architecture

See the architecture diagram in Section 2. VirtualBox Host-Only network segment: `192.168.56.0/24`.

Appendix: Full Timeline

Date / Time (UTC)	Event
2026-09-14, 17:19:43 *(time zone not recorded)*	Initial compromise (Samba usermap_script / CVE-2007-2447)
2026-09-14, 21:51:42 *(time zone not recorded)*	Persistence + C2 established (cron + beacon script)
2026-09-20, 23:05:00	First raw Suricata alert for `sid:1000001` (Wazuh rule 86601)
2026-09-20, 23:28:05	Remediation performed (crontab cleaned, verified via diff + hash comparison) — Metasploitable2 clock
2026-09-20, 23:40:06	First detection (Wazuh rule 100101) — after remediation as recorded, see Section 8
2026-09-21, 21:51:44	Second detection (Wazuh rule 100101) — last alert observed
2026-09-22, 19:02	End of the monitoring window, no further alerts